

Housing Dataset Analysis

RAW FILE :

Housing.csv

DATA CLEANING :

SOFTWARES USED FOR DATA CLEANING :

- EXEL
- DATA WRANGLER
- JUPITER NOTEBOOK
- PYTHON LIBRARY PANDAS

CODE :

```
# include necessary libraries
import numpy as np
import pandas as pd

# load dataset
data = pd.read_csv('housing.csv')

# convert price to millions
data['price(millions)'] = data['price'] / 1000000

# log transformation of price
data['log_price'] = np.log(data['price'])
```

```

# Encoding columns
cols = ["mainroad", "guestroom", "basement",
        "hotwaterheating", "airconditioning", "prefarea"]

data[cols] = data[cols].replace({"no": 0, "yes": 1})

# for selecting numeric data
numeric_data = data.select_dtypes(include=['float64', 'int64'])

# minimal information about data
data.info()
data.head(3)
data.describe()

```

MACHINE LEARNING MODELS :

1. LINEAR REGRESSION MODEL

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error, mean_squared_error

# load csv data
data = pd.read_csv("Housing.csv")

# feature engeneering
# for log transformation
data['log_price'] = np.log(data['price'])

cols = ["mainroad", "guestroom", "basement",

```

```

    "hotwaterheating", "airconditioning" , "prefarea"]

data[cols] = data[cols].replace({"no": 0 , "yes": 1})

# Seperate features and target
X = data.drop(["price", "log_price"], axis=1)
Y = data["log_price"]

# one_hot encoding
X = pd.get_dummies(X , columns=['furnishingstatus'], drop_first=True)

# test-train split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_
state=42)

# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# train linear regression
model = LinearRegression()
model.fit(X_train, Y_train)

#predictions
Y_train_pred = model.predict(X_train)
Y_test_pred = model.predict(X_test)

# calculate r2 score
r2 = r2_score(Y_test, Y_test_pred)
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)

# print values
print("R2 score : " , r2)

```

```
print("MAE:", mae)
print("RMSE:", rmse)
```

2. RIDGE REGRESSION MODEL :

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge as ridge_regression
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error, mean_squared_error

# load csv data
data = pd.read_csv("Housing.csv")

# feature engeneering
# for log transformation
data['log_price'] = np.log(data['price'])

cols = ["mainroad", "guestroom", "basement",
        "hotwaterheating", "airconditioning", "prefarea"]

data[cols] = data[cols].replace({"no": 0, "yes": 1})

# Seprate features and target
X = data.drop(["price", "log_price"], axis=1)
Y = data["log_price"]

# one_hot encoding
X = pd.get_dummies(X, columns=['furnishingstatus'], drop_first=True)

# test-train split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_
state=42)
```

```

# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# train ridge regression
# multiple values of alpha can be tested for better results [0.01, 0.1, 1, 10, 100]
# among all 0.01 gives the best results
model = ridge_regression(alpha=0.01)
model.fit(X_train, Y_train)

#predictions
Y_train_pred = model.predict(X_train)
Y_test_pred = model.predict(X_test)

# calculate r2 score
r2 = r2_score(Y_test, Y_test_pred)
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)

# print values
print("R2 score : ", r2)
print("MAE:", mae)
print("RMSE:", rmse)

```

3. LASSO REGRESSION MODEL :

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error, mean_squared_error

```

```

# load csv data
data = pd.read_csv("Housing.csv")

# feature engeneering
# for log transformation
data['log_price'] = np.log(data['price'])

cols = ["mainroad", "guestroom", "basement",
        "hotwaterheating", "airconditioning", "prefarea"]

data[cols] = data[cols].replace({"no": 0, "yes": 1})

# Seperate features and target
X = data.drop(["price", "log_price"], axis=1)
Y = data["log_price"]

# one_hot encoding
X = pd.get_dummies(X, columns=['furnishingstatus'], drop_first=True)

# test-train split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_
state=42)

# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# train lasso regression
model = Lasso(alpha=0.01, max_iter=10000)
model.fit(X_train, Y_train)

#predictions
Y_train_pred = model.predict(X_train)
Y_test_pred = model.predict(X_test)

# calculate r2 score

```

```

r2 = r2_score(Y_test, Y_test_pred)
mae = mean_absolute_error(Y_test, Y_test_pred)
mse = mean_squared_error(Y_test, Y_test_pred)
rmse = np.sqrt(mse)

# print values
print("R2 score : " , r2)
print("MAE:", mae)
print("RMSE:", rmse)

# Feature selection check
feature_names = X.columns
coefficients = pd.Series(model.coef_, index=feature_names)

print("Selected Features (Non-zero coefficients):")
print(coefficients[coefficients != 0].sort_values())

```

4. RANDOM FOREST REGRESSION MODEL :

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Load dataset
data = pd.read_csv("Housing.csv")

# Log transform target
data['log_price'] = np.log(data['price'])

# Binary encoding
cols = ["mainroad", "guestroom", "basement",
        "hotwaterheating", "airconditioning", "prefarea"]

```

```

data[cols] = data[cols].replace({"no": 0, "yes": 1})

# Features & target
X = data.drop(["price", "log_price"], axis=1)
y = data["price"]

# One-hot encode furnishing status
X = pd.get_dummies(X, columns=["furnishingstatus"], drop_first=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Random Forest model
rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)

# Train model
rf.fit(X_train, y_train)

# Predictions
y_pred = rf.predict(X_test)

# Metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

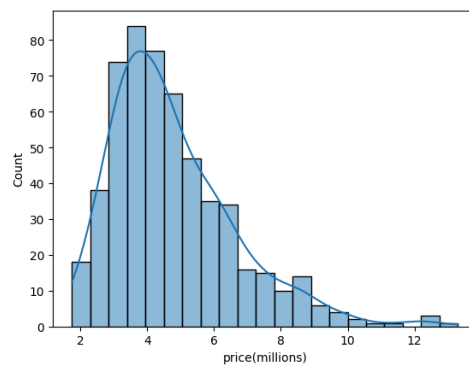
print("Random Forest Results")
print("R2 Score :", r2)

```

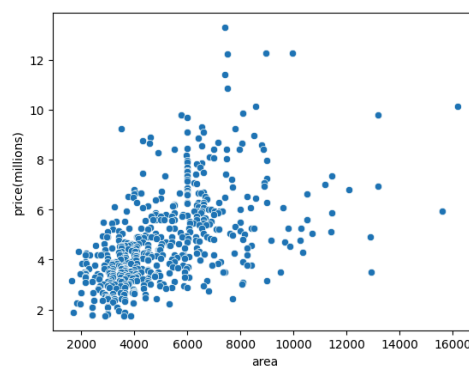
```
print("MAE   :", mae)
print("RMSE  :", rmse)
```

VISUALIZATION CODE :

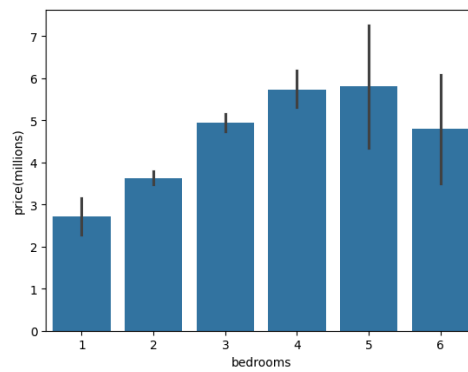
```
# price distribution
sb.histplot( data['price(millions)'], kde=True)
```



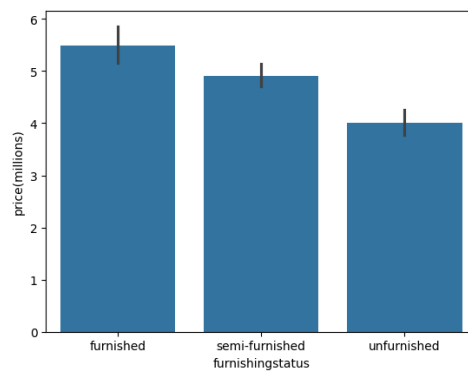
```
# scatter plot between price and area
sb.scatterplot(x='area', y='price(millions)', data=data)
```



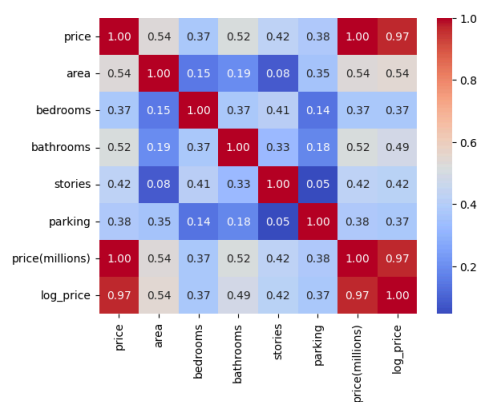
```
# correlation between bedrooms and price
sb.barplot(x='bedrooms', y='price(millions)', data=data)
```



```
# box plot between furnishing status and price
sb.barplot( x='furnishingstatus', y='price(millions)', data=data)
```



```
# heatmap for numeric data
corr = numeric_data.corr()
sb.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
```



Housing Price Prediction (Machine Learning Project)

Project Overview

This project focuses on predicting **house prices** using machine learning techniques.

Multiple regression models were implemented, evaluated, and compared to identify the most suitable approach based on data characteristics and performance.

Dataset

- **Name:** Housing.csv
- **Rows:** 545
- **Columns:** 13
- **Target Variable:** `price`

Feature Types

- **Numerical:** area, bedrooms, bathrooms, stories, parking, price
 - **Binary (Yes/No):** mainroad, guestroom, basement, hotwaterheating, airconditioning, prefarea
 - **Categorical:** furnishingstatus (furnished, semi-furnished, unfurnished)
-

Exploratory Data Analysis (EDA)

Key Observations

- No missing values in the dataset
- Price distribution is **right-skewed**
- Strong positive correlation between:
 - `price` and `area`
 - `price` and `bathrooms` , `bedrooms`
- Furnished houses tend to have higher prices
- Houses with amenities (`hotwaterheating` , `prefarea` , `basement`) are priced higher

Visualizations Performed

- Price distribution (histogram + KDE)
 - Area vs Price scatter plot
 - Bedrooms vs Price
 - Furnishing status vs Price
 - Correlation heatmap (numerical features)
-

Feature Engineering & Preprocessing

Target Transformation

- Log transformation applied:

```
data['log_price'] = np.log(data['price'])
```

Used to:

- Reduce skewness
- Improve linear model performance

Encoding

- Binary features encoded as:

```
yes → 1  
no → 0
```

- One-hot encoding for `furnishingstatus` :

```
pd.get_dummies(drop_first=True)
```

Scaling

- StandardScaler applied for linear models

- Not used for tree-based models
-

Models Implemented

1 Linear Regression

- Baseline model
- Performance limited by multicollinearity

$R^2 \approx 0.67$

2 Ridge Regression (Best Model)

Used to handle multicollinearity and stabilize coefficients.

Best alpha: 0.01 – 0.1

Performance:

- $R^2 \approx 0.67$
- MAE ≈ 0.20 (log scale)
- RMSE ≈ 0.25 (log scale)

 Chosen as final model

3 Lasso Regression

- Performed feature selection
- Slight underfitting observed

$R^2 \approx 0.65$

Important Selected Features:

- area
 - bathrooms
 - airconditioning
 - stories
 - prefarea
-

4 Random Forest Regression

Tested to capture non-linear relationships.

Result:

- R^2 dropped to **~0.61**
- Higher MAE & RMSE
- Poor generalization



Reason for failure:

- Small dataset
- Mostly linear relationships
- Correlated features
- Tree-based models overfit noise



Model Comparison

Model	R^2 Score
Linear Regression	~0.67
Ridge Regression	~0.67 (Best)
Lasso Regression	~0.65
Random Forest	~0.61



Key Learnings

- Model complexity does not guarantee better performance
- Ridge regression is well-suited for small, linear datasets
- Random Forests require larger datasets and non-linear patterns
- Proper EDA and diagnostics are crucial for model selection



Final Conclusion

Ridge Regression with log-transformed price is the most suitable model for this dataset due to its stability, interpretability, and superior generalization performance.

Next Steps

- Residual diagnostics and assumption checks
 - Polynomial and interaction features
 - Apply workflow to a more complex dataset
-

Tech Stack

- Python
 - Pandas, NumPy
 - Seaborn, Matplotlib
 - Scikit-learn
-

Final Note

This project demonstrates **end-to-end ML workflow**:

EDA → Feature Engineering → Model Selection → Evaluation → Documentation